The Design Process: From Data to Display

Workshop 1 — Module 6

Prof.Asoc. Endri Raco

2025/26

Data Visualization for Data Scientists
UNYT Tirana

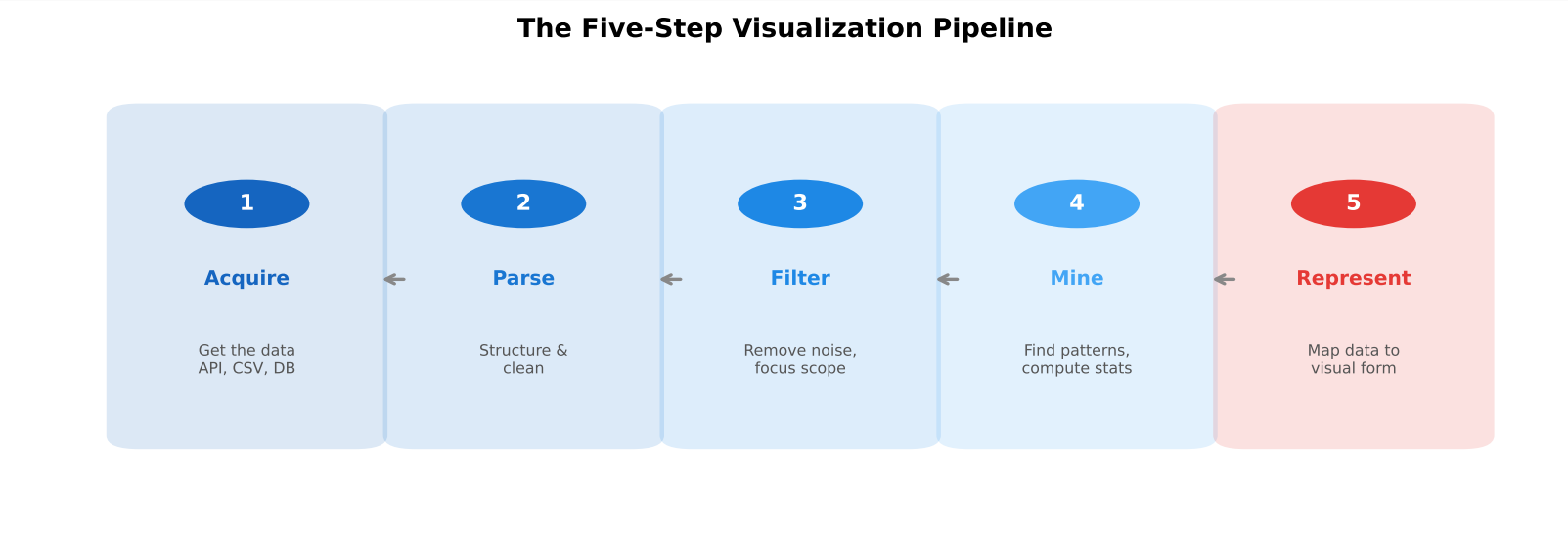
Module Roadmap

1. The visualization pipeline: acquire $\rightarrow$ represent $\rightarrow$ refine
2. Cairo's visualization wheel and design trade-offs
3. Munzner's nested model: why $\rightarrow$ what $\rightarrow$ how
4. Audience analysis and chart-type selection
5. Iterative refinement: from sketch to publication

Learning Outcome
You will be able to articulate a systematic design process that begins with the audience's question and ends with a polished, principled chart.

The Visualization Pipeline

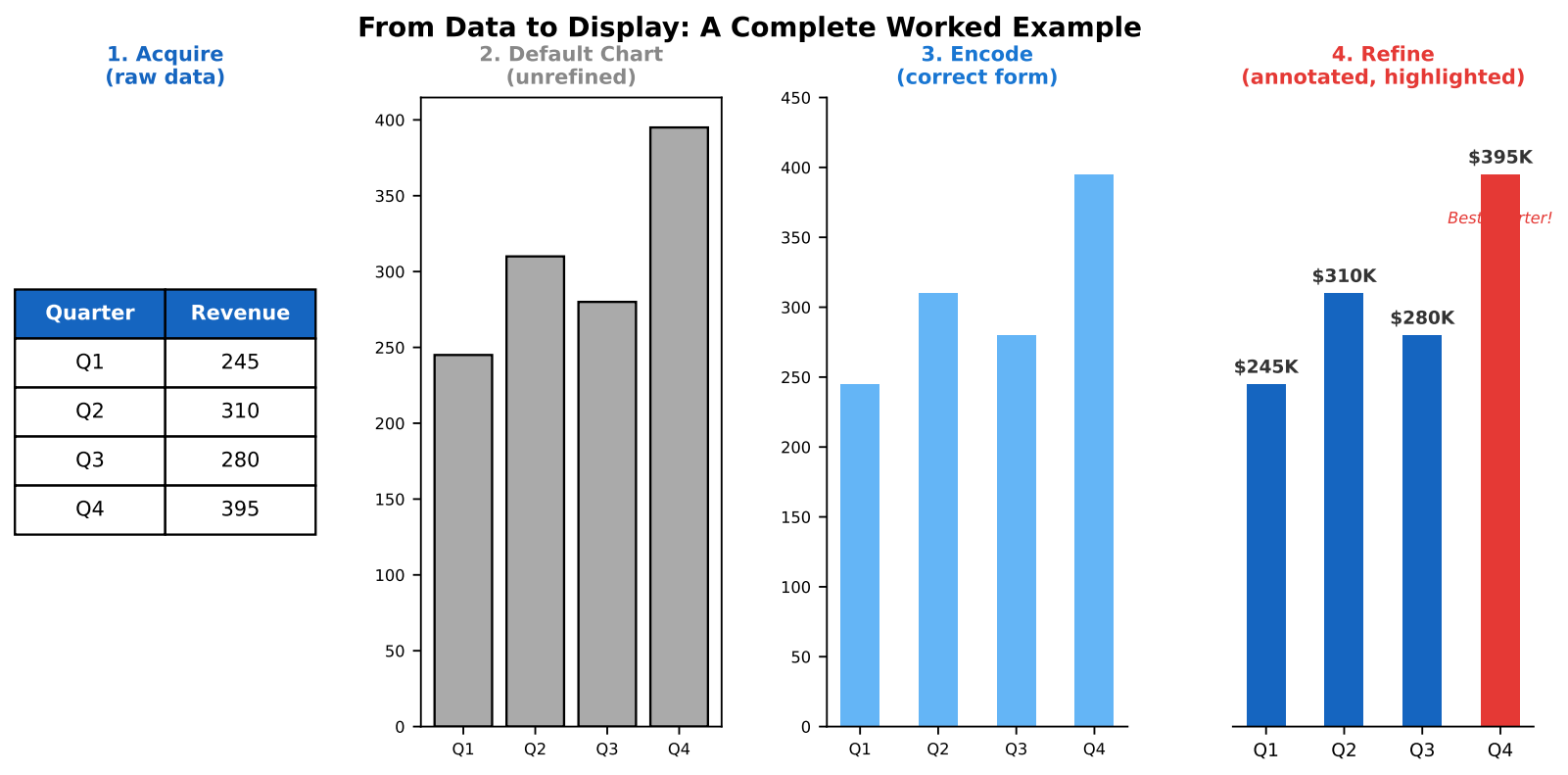
Five Steps: Acquire → Parse → Filter → Mine → Represent



Pro-Tip

Most analysts jump directly to Step 5 (Represent). Steps 1–4 consume 80% of the effort but determine 80% of the chart's value. A beautiful chart built on dirty data is a beautiful lie.

From Raw Data to Final Chart: Worked Example

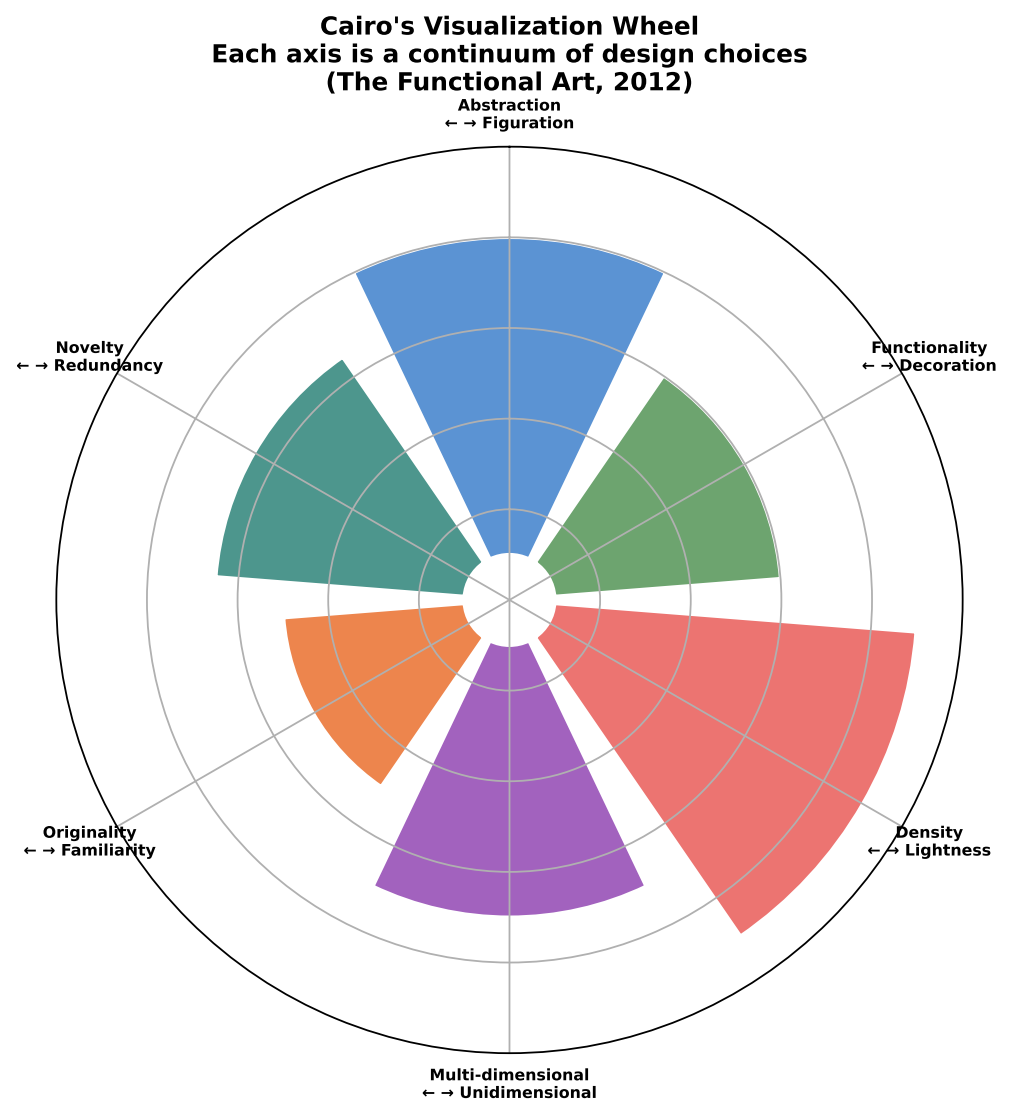


Data Insight

The same four numbers pass through four stages: raw table, default chart, correctly encoded bars, and a polished chart with direct labels and an accent colour on the highest value. Each step adds interpretive value without adding data.

Design Frameworks

Cairo's Visualization Wheel



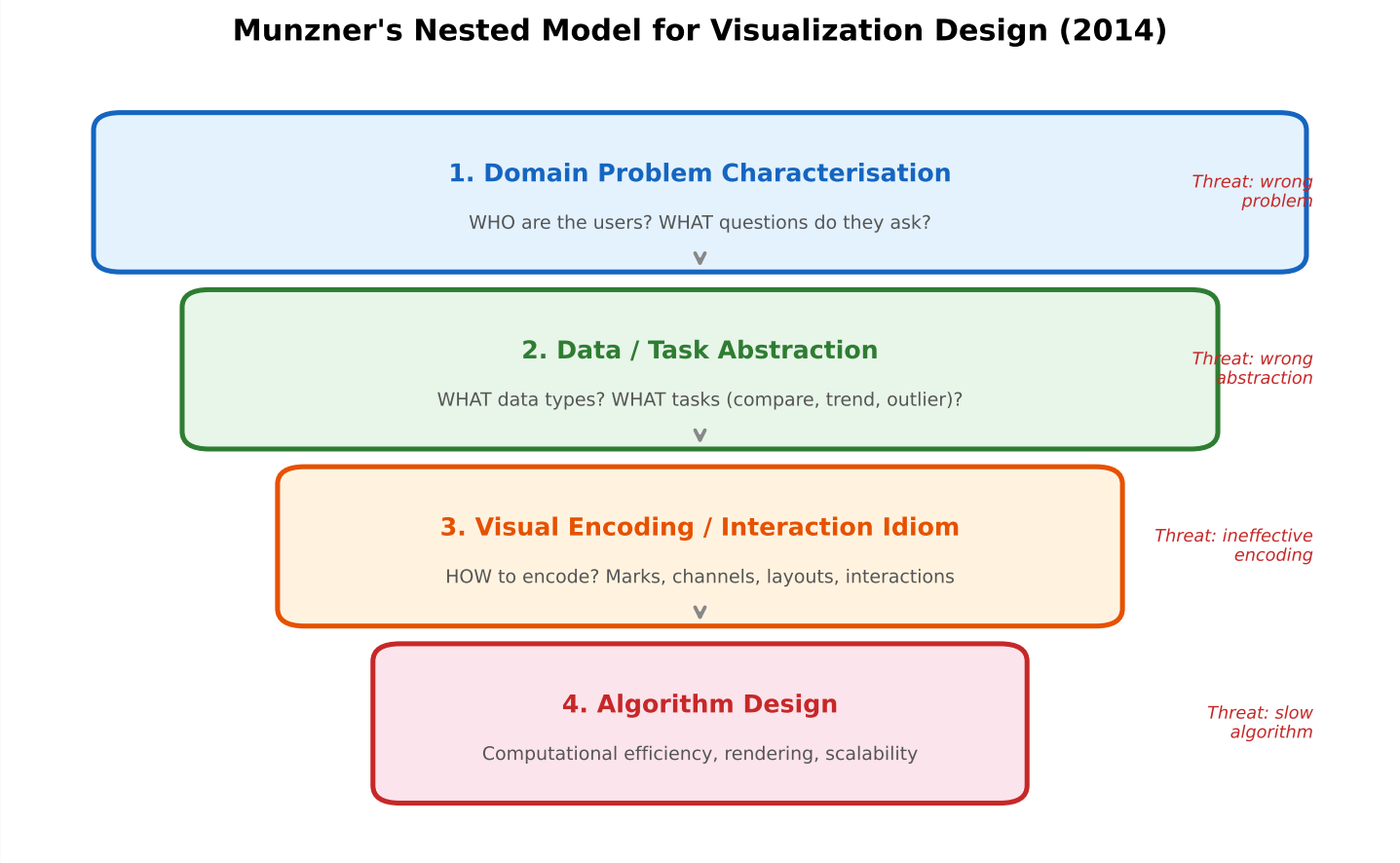
Six continuums of design tension. Every visualization sits somewhere on each axis; there is no single "correct" position — only positions that are

Cairo's Six Design Tensions

Axis	Left Pole	Right Pole
1	Abstraction (statistical)	Figuration (pictorial)
2	Functionality (analytical)	Decoration (aesthetic)
3	Density (data-rich)	Lightness (minimal)
4	Multidimensional	Unidimensional
5	Originality (novel form)	Familiarity (standard form)
6	Novelty (surprising)	Redundancy (reinforcing)

Pro-Tip
For analytical audiences (analysts, scientists), lean toward abstraction, functionality, and density. For general audiences, lean toward figuration, decoration, and lightness.

Munzner's Nested Model



Nested Model: Threats at Each Layer

1. **Domain layer**: solving the *wrong problem* — the chart answers a question nobody asked
2. **Abstraction layer**: wrong data type mapping — treating ordinal data as nominal, or continuous as discrete
3. **Encoding layer**: ineffective visual idiom — using a pie chart for 20 categories, or area for precise comparisons
4. **Algorithm layer**: slow rendering for large data — plotting 10M points without aggregation or WebGL

Data Insight

Errors at outer layers propagate inward. No amount of visual polish (layer 3) can fix a chart that answers the wrong question (layer 1). **Always validate outward-in.**

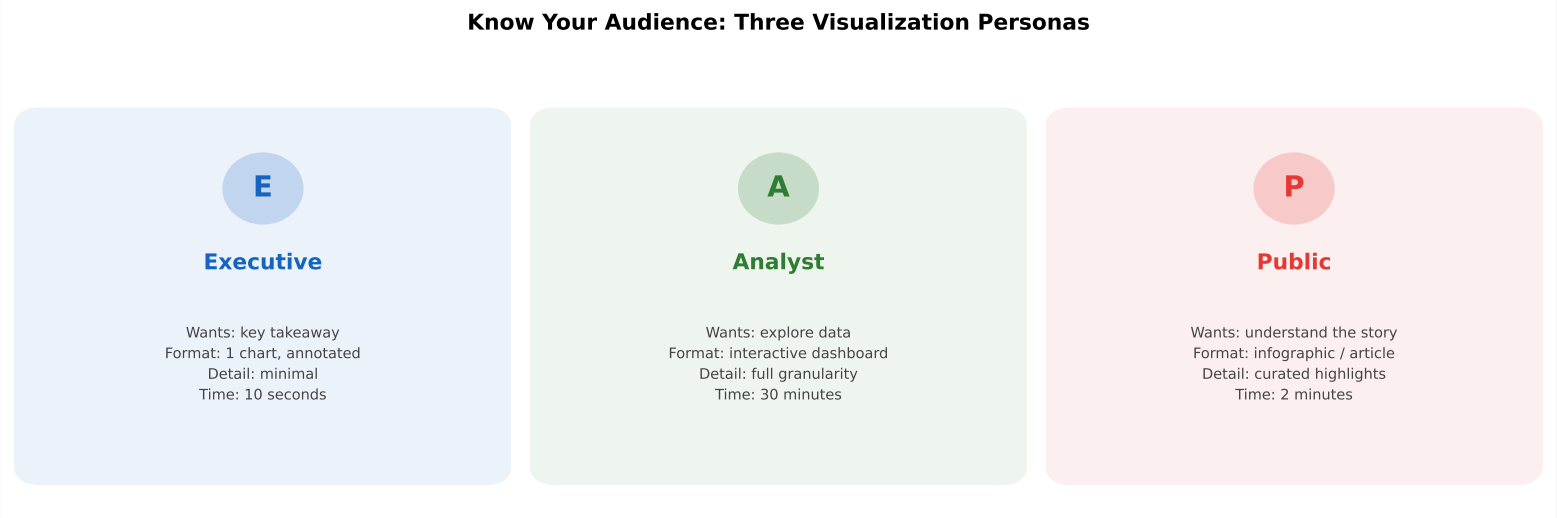
Task Taxonomy: Why × What × How

Munzner's Task Taxonomy: Why × What × How

WHY (Task)	WHAT (Data)	HOW (Idiom)
Discover — find new patterns	Tables — rows × columns	Encode — marks + channels
Present — communicate known findings	Networks — nodes + edges	Manipulate — select, navigate, arrange
Enjoy — casual browsing / exploration	Spatial — geographic / field data	Facet — juxtapose, partition, layer
	Temporal — time-indexed sequences	Reduce — aggregate, filter, sample

Audience & Chart Selection

Know Your Audience



Audience Drives Every Design Decision

Decision	Executive	Analyst
Detail level	1 key metric	Full dataset
Interactivity	Static, annotated	Filters, drill-down
Palette	Grey + 1 accent	Multi-hue qualitative
Chart count	1 per slide	Dashboard grid
Annotation	Bold callout	Reference lines

Pro-Tip
Ask three questions before opening your tool: (1) Who will read this? (2) What decision will they make? (3) How much time will they spend?

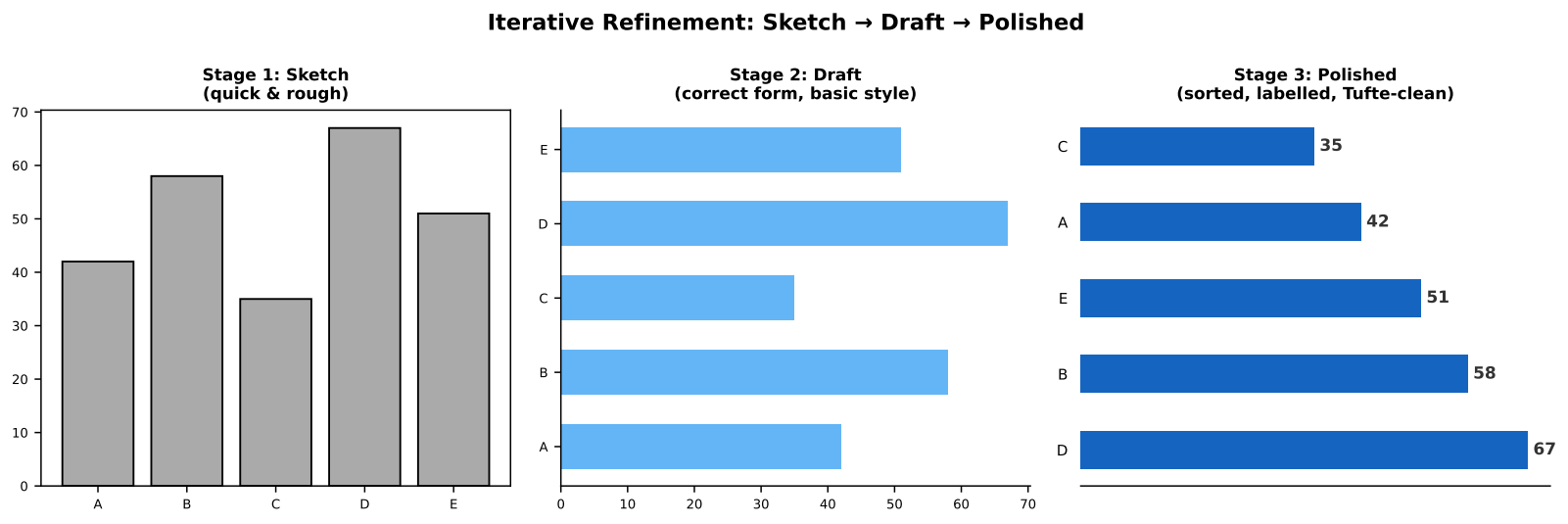
Chart Selection Matrix

Chart Selection: Match the Analytical Goal to the Visual Form

Comparison	**Bar, Dot, Lollipop** across categories
Distribution	**Histogram, Density, Boxplot** of a single variable
Relationship	**Scatter, Bubble, Hexbin** between two+ variables
Composition	**Stacked Bar, Treemap, Waffle** parts of a whole
Trend	**Line, Area, Sparkline** change over time
Ranking	**Horizontal Bar, Bump, Slope** ordered categories
Spatial	**Choropleth, Point Map, Hexbin** geographic patterns

Iterative Refinement

Sketch → Draft → Polished



Data Insight

Iteration is not optional. Even Tufte's exemplars went through multiple drafts. Start rough (sketch the structure), then refine the encoding (draft), then polish the typography and colour (final).

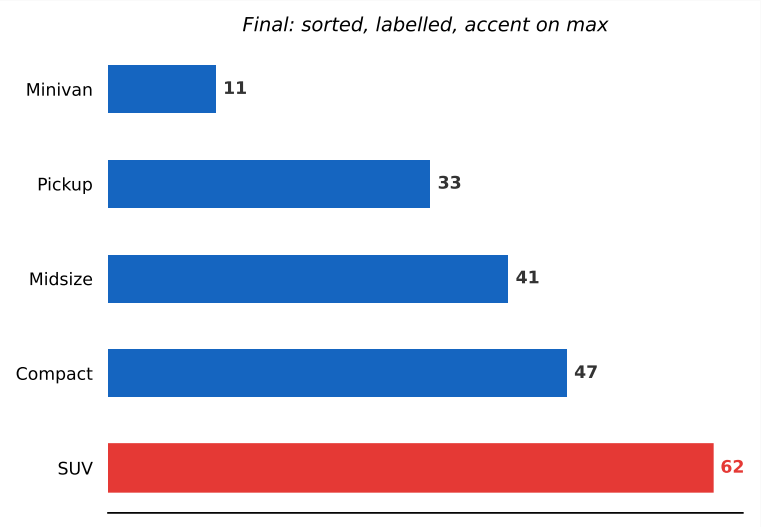
Iterative Refinement in R

```r
library(tidyverse)

# Step 1: Quick sketch
ggplot(df, aes(x = cat, y = val)) +
  geom_col()  # default everything

# Step 2: Correct form
ggplot(df,
  aes(x = fct_reorder(cat, val),
      y = val)) +
  geom_col(fill = "#64B5F6",
           width = 0.6) +
  coord_flip()

# Step 3: Polished
ggplot(df,
  aes(x = fct_reorder(cat, val),
      y = val)) +
  geom_col(fill = "#1565C0",
           width = 0.5) +
  geom_text(aes(label = val),
    hjust = -0.2, size = 3,
    fontface = "bold") +
  coord_flip() +
  theme_minimal() +
  theme(panel.grid.major.y =
```



Three commits:

1. `geom_col()` — see the data
2. `coord_flip()` + colour — correct the form
3. Theme + labels + sort — publication quality

Iterative Refinement in Python

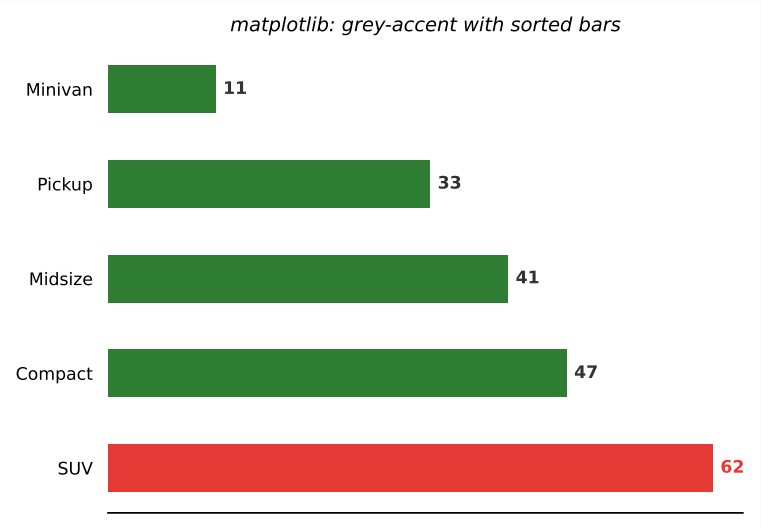
```python
import matplotlib.pyplot as plt

cats = ["A","B","C","D","E"]
vals = [42, 58, 35, 67, 51]

# Step 1: Quick sketch
fig, ax = plt.subplots()
ax.bar(cats, vals)  # defaults

# Step 2: Correct form
fig, ax = plt.subplots()
idx = sorted(range(len(vals)),
    key=lambda i: vals[i])
ax.barh([cats[i] for i in idx],
    [vals[i] for i in idx],
    color="#64B5F6", height=0.6)

# Step 3: Polished
fig, ax = plt.subplots()
colors = ["#1565C0"] * 5
colors[-1] = "#E53935"  # max
ax.barh([cats[i] for i in idx],
    [vals[i] for i in idx],
    color=[colors[i] for i in idx],
    height=0.5)
for i, v in enumerate(
```



Same three steps:

1. `ax.bar()` — default
2. Sort + horizontal — correct the form
3. Accent colour + direct labels + spine removal

The Complete Pipeline in R (20 lines)

```
library(tidyverse)

# 1. ACQUIRE
df <- read_csv("googleplaystore.csv")

# 2-3. PARSE + FILTER
df_clean <- df |>
  filter(Type %in% c("Free","Paid"))
      |>
  count(Category, sort = TRUE) |>
  slice_max(n, n = 10) |>
  mutate(Category =
    fct_reorder(Category, n))

# 4-5. MINE + REPRESENT
ggplot(df_clean,
  aes(x = Category, y = n)) +
  geom_col(fill = "#1565C0",
           width = 0.6) +
  geom_text(aes(label = n),
    hjust = -0.2, size = 3) +
  coord_flip() +
  theme_minimal() +
  theme(panel.grid.major.y =
    element_blank()) +
  labs(title = "Top 10 App Categories
```

Pipeline in 20 lines:

Lines 1–3: Acquire (read CSV)
Lines 5–10: Parse + Filter (clean types, count, top 10, reorder)
Lines 12–23: Mine + Represent (bar chart with Tufte styling)

Pro-Tip
The pipe operator (|>) makes the pipeline explicit: each verb is one step in the design process.

The Complete Pipeline in Python (25 lines)

```python
import pandas as pd
import matplotlib.pyplot as plt

# 1. ACQUIRE
df = pd.read_csv(
    "googleplaystore.csv")

# 2-3. PARSE + FILTER
df_clean = (df
    .query("Type in ['Free','Paid']")
    .value_counts("Category")
    .head(10)
    .sort_values()
    .reset_index(name="n"))

# 4-5. MINE + REPRESENT
fig, ax = plt.subplots(figsize=(7,5))
ax.barh(df_clean["Category"],
    df_clean["n"],
    color="#1565C0", height=0.6)
for i, v in enumerate(df_clean["n"]):
    ax.text(v+5, i, str(v),
        va="center", fontsize=8)
ax.set_title("Top 10 App Categories",
    fontweight="bold")
ax.spines["top"].set_visible(False)
```

Pipeline in 25 lines:

Lines 1–5: Acquire
Lines 7–12: Parse + Filter (method chaining mirrors the pipe)
Lines 14–25: Mine + Represent

Method chaining in pandas (`.query().value_counts().head().sort_values()`) is the Python equivalent of R's pipe.

Synthesis

The Design Process Checklist

Before creating any chart, complete this checklist:

1. **Who** is the audience? (Executive / Analyst / Public)
2. **What** question does the chart answer?
3. **What** data type? (Table / Network / Spatial / Temporal)
4. **What** analytical goal? (Compare / Distribute / Relate / Compose / Trend)
5. **How** — choose the chart type from the decision matrix
6. **Iterate** — sketch $\rightarrow$ draft $\rightarrow$ polished
7. **Validate** — check against Munzner's four threats

Key Takeaways

1. The **five-step pipeline** (acquire $\rightarrow$ parse $\rightarrow$ filter $\rightarrow$ mine $\rightarrow$ represent) ensures that data quality precedes visual design.
2. **Cairo's wheel** frames design as a set of trade-offs, not a single optimum. The audience and purpose determine the position.
3. **Munzner's nested model** identifies four layers of potential failure. Always validate outward-in.
4. **Chart selection** follows from the analytical goal: comparison $\rightarrow$ bar; distribution $\rightarrow$ histogram; relationship $\rightarrow$ scatter.
5. **Iteration** is mandatory: sketch (see the data), draft (correct the form), polished (publication quality).

Essential Readings for This Module

- Cairo, A. (2012). *The Functional Art*, Chapters 2–3 (The Visualization Wheel, Choosing the Right Form).
- Munzner, T. (2014). *Visualization Analysis and Design*, Chapters 1–4 (Nested Model, Task Taxonomy).
- Kirk, A. (2019). *Data Visualisation: A Handbook for Data Driven Design*, Chapters 4–6 (The Design Process).
- Schwabish, J. (2021). *Better Data Visualizations*, Chapter 2 (The Chart Chooser).

Next Module: Critique & Redesign Workshop (W01-M07)